

Design: WFP connect-redirect callout — the Windows equivalent of `NETtransparentProxyProvider`

2026-10-02. Windows counterpart to `copilot-desktop-nettransparentproxyprovider-design.md`: the same problem (durable, launch-method-agnostic interception of the Copilot desktop app's traffic, replacing the one-shot `HTTPS_PROXY` env var proven tonight), solved with Windows's own redirect-capable primitive instead of a Network Extension.

The good news: most of the surrounding infrastructure already ships

Unlike the macOS side — where essentially the whole extension is net-new — `network_egress`'s existing, production Windows implementation (`docs/network-egress.md`) already has every piece *except* the actual redirect action:

- **Process-scoped matching**, via `FWPM_CONDITION_ALE_APP_ID` derived from the connecting process's image path (`FwpmGetAppIdFromFileName0`) — the exact role macOS's `NEAppRule` plays, already real and enforced today for block/allow rules.
- **DNS-to-IP correlation** (`reconcile::compute_filters`), turning a `domains` rule into resolved destination filters as the ETW sensor learns them.
- The whole settings-delivery pipeline, policy schema, and dynamic-session filter bookkeeping.

So the actual new-work surface here is narrower than the macOS document's: **just the callout that performs the redirect**, reusing the process/DNS matching that already exists.

Why this needs a callout, not another filter

`network_egress`'s current block/allow rules are pure **filters** — a declarative condition (app id, destination, port, user) paired with one of WFP's own *built-in* actions (permit/block). No custom code runs; WFP's engine evaluates the condition and applies the stock action itself. That's exactly why today's Windows enforcer has **no kernel driver at all** ("the enforcer uses the user-mode `Fwpm*` management APIs").

A **callout** is different: custom classify-function code that WFP invokes for traffic matching a filter that references it, which decides the outcome dynamically. Redirect isn't one of WFP's built-in filter actions — it requires a callout, and specifically one registered on a different layer from the one `network_egress` uses today.

The layer, and what it actually does

- `network_egress` observes/blocks at `FWPM_LAYER_ALE_AUTH_CONNECT_V4` — fires when a connection is about to be authorized; can only permit or deny.
- Redirect lives at `FWPM_LAYER_ALE_CONNECT_REDIRECT_V4` — fires earlier in the `connect()` sequence, specifically so a callout can substitute a *different* destination address/port before the SYN is sent. The OS

then completes the handshake against the new address. This is what makes it transparent to the calling app: it asked to connect to the real host, the kernel silently connects it elsewhere instead, and the app generally has no simple way to tell.

- Confirmed from Microsoft's own WFP layer documentation: available since Windows 7, redirection is scoped to the individual flow (not the whole socket), and callouts targeting this layer must register with `FwpsCalloutRegister1` or later — `FwpsCalloutRegister0` does not work here.

Architecture — narrower in scope than the macOS extension

This is the one place the two platforms genuinely diverge in design, not just implementation:

```
GitHub Copilot.exe (any launch method)
  | connect() to api.business.githubcopilot.com
  ▼
WFP callout @ FWPM_LAYER_ALE_CONNECT_REDIRECT_V4
  | FWPM_CONDITION_ALE_APP_ID matches: source is GitHub Copilot.exe
  | callout substitutes the destination: 127.0.0.1:<listener port>
  ▼
TCP handshake completes against the LOCAL LISTENER, not the real host
  | (ordinary userspace process from here on – the kernel's job is done)
  ▼
ai-gateway's listener – same TLS-terminating, model-list-faking,
real-bearer-pass-through logic already designed for the macOS side
```

On macOS, `NETransparentProxyProvider` hands the extension the entire flow object — the extension process itself can read/write the connection's bytes, so the TLS/HTTP logic could in principle live inside the extension. The WFP redirect callout gets none of that: its only job is changing *where the connection lands*. Once redirected, it's an ordinary TCP connection to an ordinary process. **All the actual interception logic — TLS termination, faking `GET /models`, passing the app's own real bearer through for everything else — stays in the same userspace listener already designed for this**, not in the kernel. That's a materially simpler scope for the kernel-mode component than the macOS extension carries, even though the *signing* story (below) is harder.

What shipping this actually requires

- **A new kernel-mode driver.** Not an extension of anything existing — `network_egress`'s Windows side is explicitly driver-free today, built entirely on user-mode `Fwpm*` calls. This is the first kernel-mode component this feature would ship.
- **Driver signing**, which is a harder and more independent gate than macOS's approval flow. A kernel-mode driver on modern Windows needs an EV code-signing certificate plus Microsoft attestation signing (or full WHQL for wider distribution) to load at all under driver-signature enforcement. There is no analogue to macOS's "MDM can silently pre-approve the one-time user click" — Windows driver signing isn't something an MDM profile can substitute for or bypass.
- **A harder crash-safety bar.** A bug in this callout's classify function can bluescreen the machine. A bug in the macOS extension (which runs in user space — Apple deprecated kernel extensions for exactly this class of work) just kills that one extension process. Not a detail to gloss over: the two platforms are not symmetric in blast radius for equivalent functionality.

- **The exact WDK API surface needs verification before any code is written** — the registration call (`FwpsCalloutRegister1` +) is confirmed from Microsoft's documentation, but the specific redirect-record structures/functions used inside the `classify` function should be checked against current WDK documentation rather than assumed from memory; this is kernel code, where a subtly wrong symbol is expensive to get wrong.

An inherited caveat, not a new one

`FWPM_CONDITION_ALE_APP_ID` needs the ETW sensor to have already attributed the connecting PID to its image path before a filter referencing it can match — `network_egress` 's own documentation already states this plainly: "a constrained rule whose process has not been observed yet... simply waits until the mapping exists." A redirect rule inherits the exact same gap: the very first connection attempt from a freshly-launched Copilot process could race ahead of that attribution and momentarily miss the redirect. This isn't new risk introduced by adding redirect — it's a property of infrastructure already shipping for block/allow — but it applies here identically and shouldn't be assumed away.

Platform comparison, at a glance

	macOS (<code>NETransparentProxyProvider</code>)	Windows (WFP redirect callout)
Runs in	User space (system extension)	Kernel mode (driver)
Crash blast radius	One extension process	Whole machine (bluescreen)
Signing	Developer ID + notarization + one-time approval (MDM can pre-approve)	EV cert + attestation/WHQL — MDM cannot substitute
App-scoping mechanism	<code>NEAppRule</code> (bundle id)	<code>FWPM_CONDITION_ALE_APP_ID</code> (image path) — already shipping for block/allow
Reused existing infra	None — new extension target entirely	Process attribution, DNS correlation, policy schema, settings pipeline — all already shipping
What the OS component itself does	Can proxy the full flow in-process	Only changes the destination; a separate userspace listener does everything else

Recommended build order

1. **Verify the exact classify-function API surface** against current WDK docs before writing any driver code — this gates everything else.
2. **Decide the signing path early** (attestation signing vs. full WHQL) — this has its own lead time independent of the code itself and should not be discovered late.
3. Build the callout to do **only** the redirect (app-id match → `127.0.0.1:<port>`), reusing `network_egress` 's existing process/DNS matching rather than reimplementing it.
4. Point the redirected connections at the **same local listener** already designed for the macOS side — the `/models` -faking and real-bearer pass-through logic should be written once and reused by both platforms, not duplicated.

5. Validate against the inherited process-attribution race explicitly (fresh-launch timing), since it's a known gap in the infrastructure this reuses, not a hypothetical.